Training models

Gradient descent, and the models it fits

LECTURE 4 | GÉRON CH. 4

How a model is *fitted*, when there is no formula for the answer

---

Applications of Machine Learning

BSc Mathematics of Artificial Intelligence · Third year

Worked example: the Titanic passenger manifest

Where we are

Lecture 3 solved least squares in closed form — the **normal equation** $\hat{\boldsymbol{\theta}} = (\mathbf{X}^\intercal \mathbf{X})^{-1} \mathbf{X}^\intercal y$, defined exactly when $\mathbf{X}$ has full column rank — and then measured a classifier properly: precision, recall, and a threshold chosen against a stated constraint. Both called `.fit()` and moved on. Today we open it, because the closed form runs out:

Situation	What the normal equation does
Many features	inverts an $(n+1) \times (n+1)$ matrix — cost grows like n^3
Many instances	forms $\mathbf{X}^\intercal \mathbf{X}$, so every row must be seen at once
Rank-deficient design	the inverse does not exist
✗ A cost that is not squared error	**✗ there is no closed form at all**

The last row is the one that matters: the classifier we build today has no formula for its answer, so the answer has to be *walked to*.

Today

1. **The mathematics** — gradient descent: the gradient, the update, choosing the step size, why the features must be scaled, and why one row at a time is legitimate
2. **The method** — descent against the closed form, and polynomial features
3. **The worked example** — logistic regression on 891 Titanic passengers: a probability, not a label; a design matrix that is rank-deficient; a degree sweep; and whether the probabilities are calibrated
4. More than two classes: softmax regression

The derivation, and logistic and softmax regression: the model, the log loss, coefficients as log-odds, and why a rank-deficient design leaves the fit without a unique answer. The Titanic feature engineering.

EXAMINABLE

NOT EXAMINABLE

Open Lecture 4 in Colab

The question

You have a cost $J(\boldsymbol{\theta})$ and no formula for its minimiser.

How do you find one, and how do you know you have?

> **THREE THINGS THE ANSWER MUST SUPPLY**
>
> A **direction** to move in, a **step size** that does not overshoot, and a **reason** to believe the point you stop at is the minimum and not merely a place where you stopped.

Step 1 · The cost is a surface

m instances, n features; $\mathbf{X}$ is $m \times (n+1)$ with a leading column of ones, and $\boldsymbol{\theta} \in \mathbb{R}^{n+1}$ includes the bias. We derive on squared error, the one cost whose minimiser we can also compute in closed form — so the method can be *checked*:

$$J(\boldsymbol{\theta}) \;=\; \mathrm{MSE}(\boldsymbol{\theta}) \;=\; \frac{1}{m}\,\|\mathbf{X}\boldsymbol{\theta} - y\|_2^2$$

> **THE ALGORITHM, IN ONE SENTENCE**
>
> Start anywhere. Step **downhill**, along $-\nabla J$. Repeat until the downhill direction is flat.

Why $-\nabla J$: along a unit direction $\mathbf{u}$ the cost changes at rate $\nabla J \cdot \mathbf{u}$, and by Cauchy–Schwarz that is most negative when $\mathbf{u}$ points along $-\nabla J$.

Step 2 · The gradient

$$\nabla J(\boldsymbol{\theta}) \;=\; \frac{2}{m}\,\mathbf{X}^{\mathsf{T}}\big(\mathbf{X}\boldsymbol{\theta} - y\big)$$

THE GRADIENT OF THE MEAN SQUARED ERROR

The chain rule on the square gives one partial derivative, $\partial J/\partial\theta_j = \frac{2}{m}\sum_i \big(\boldsymbol{\theta}^{\mathsf{T}}\mathbf{x}^{(i)} - y^{(i)}\big)x_j^{(i)}$ — **error × feature**, averaged. That sum is the j-th entry of $\mathbf{X}^{\mathsf{T}}\mathbf{r}$, with $\mathbf{r} = \mathbf{X}\boldsymbol{\theta} - y$ the residual vector.

✓ One matrix product over the whole training set. Cost per evaluation: $O(mn)$ — linear in the rows, where the normal equation was cubic in the columns.

Step 3 · The update rule

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - \eta \nabla J\left(\boldsymbol{\theta}^{(k)}\right) = \boldsymbol{\theta}^{(k)} - \frac{2\eta}{m} \mathbf{X}^{\intercal}\left(\mathbf{X}\boldsymbol{\theta}^{(k)} - y\right.$$

BATCH GRADIENT DESCENT

Batch, because every step uses the whole training set. Three lines of NumPy, and no matrix is ever inverted.

η has units: it converts a gradient into a displacement in parameter space. That is why its right value depends on how the features are scaled — step 4.

How large a step?

Nothing in the update rule fixes η. It is a **hyperparameter**: you try a few values and watch the cost.

Learning rate	What you see
too small	the cost falls, slowly, and is still falling when you stop
✓ about right	**✓ the cost drops fast and flattens**
✗ too large	**✗ every step overshoots the minimum: the cost *rises*, oscillates, and overflows to `nan`**

FAILURE CONDITION — DIVERGENCE IS SILENT UNTIL IT IS LOUD

A diverging fit does not raise. It returns weights full of `inf` and a model that predicts one value everywhere. **Plot the cost against the step number**; it is the only cheap way to see which of the three rows you are in.

Step 4 · Why you must scale the features

Put one feature in thousands and another in units. The cost changes far faster along the first parameter than along the second, and the bowl of step 1 becomes a long, thin valley.

- One η serves every direction, so it must be small enough not to overshoot along the steep one
- That same η is far too small along the shallow one, where progress crawls
- The path zig-zags across the valley instead of running down it

✓ Standardising the columns is not cosmetic. It makes the bowl round, so one learning rate suits every direction and far fewer steps are needed.

Step 5 · One instance at a time

Each batch step costs $O(mn)$, which for large m is unaffordable. Draw an index i uniformly at random and use only that row:

$$\mathbf{g}^{(i)}(\boldsymbol{\theta}) \;=\; 2\big(\boldsymbol{\theta}^{\mathsf{T}}\mathbf{x}^{(i)} - y^{(i)}\big)\,\mathbf{x}^{(i)}$$

Take the expectation over the draw:

$$\mathbb{E}_i\left[\mathbf{g}^{(i)}\right] \;=\; \sum_{i=1}^{m} \frac{1}{m}\, 2\big(\boldsymbol{\theta}^{\mathsf{T}}\mathbf{x}^{(i)} - y^{(i)}\big)\mathbf{x}^{(i)} \;=\; \nabla J(\boldsymbol{\theta})$$

✓ The one-instance gradient is an unbiased estimate of the batch gradient.

What the derivation buys you

The result	What it lets you do today
$\nabla J = \frac{2}{m}\mathbf{X}^\intercal(\mathbf{X}\boldsymbol{\theta} - y)$	fit a linear model without inverting anything
η is a hyperparameter	read a rising cost curve as a step-size fault, not a bug
Unscaled features make a thin valley	know that scaling is part of the algorithm, not tidiness
✓ Unbiasedness of $\mathbf{g}^{(i)}$	**✓ use mini-batches on data that does not fit in memory**

✓ And one more: only step 2 used the fact that the cost was squared error. Swap the cost, recompute the gradient, and the algorithm is unchanged — which is how the classifier later today is fitted.

Two routes to the same $\hat{\theta}$

	Normal equation / SVD	Gradient descent
Cost in m	linear	linear, per step
Cost in n	✗ $n^{2.4}$ to n^3	**linear ✓**
Out-of-core	**✗ no — needs all rows**	**yes, with mini-batches ✓**
Scaling required	no	**✗ yes**
Hyperparameters	none	**✗ η, batch size, schedule, stopping rule**
Other costs	**✗ squared error only**	**any differentiable cost ✓**

When to prefer each. Few features and data that fits in memory: use the closed form, it has no knobs to get wrong. Many features, streaming data, or any cost that is not squared error: descent.

Curvature without leaving linearity

A straight line cannot fit a curve. Add the powers as *columns*:

$$\hat{y} \;=\; \theta_0 + \theta_1 x + \theta_2 x^2 \quad \text{is linear in } \boldsymbol{\theta},\ \text{not in } x$$

Everything derived above still applies unchanged — the gradient, the update, the case for scaling — because all of them were statements about $\boldsymbol{\theta}$.

`PolynomialFeatures(degree=d)` also generates every *cross* product, which is how a linear model represents an interaction such as "third class *and* travelling alone". The count is $\binom{n+d}{d}$ — the number of monomials of degree at most d — so "a bit more capacity" is never a small change.

The task

> **MARINE SAFETY PLANNING UNIT**
>
> “We are building an evacuation-assistance model from historical passenger manifests. We need to know, for each passenger profile, **how likely that person is to get off unaided** — and we need to be able to explain the assignment afterwards.”

What they asked for	What that forces
“how likely”	a **probability**, not a label
the number is used to allocate	it must be **calibrated**
“explain the assignment”	weights a human can **read**

✓ “This passenger survives” is not checkable on its own. “This passenger survives with probability 0.31” is: take everyone the model puts near 0.31 and count. That is why requirement 1 is not a formatting preference.

The data

```
def load_titanic():
    tarball = Path("datasets/titanic.tgz")
    if not tarball.is_file():
        Path("datasets").mkdir(parents=True, exist_ok=True)
        url = "https://github.com/ageron/data/raw/main/titanic.tgz"
        urllib.request.urlretrieve(url, tarball)
        with tarfile.open(tarball) as t:
            t.extractall(path="datasets", filter="data")
    return pd.read_csv("datasets/titanic/train.csv")
```

891 passengers, 12 columns — the whole labelled corpus (the archive's `test.csv` has no `Survived` column, so we hold out our own test set). To the class, sex, age, fare and family counts we add four engineered columns: `Title` from the name (Mr, Mrs, Miss, Master, Rare), `FamilySize = SibSp + Parch + 1`, `IsAlone`, and `Deck` — the cabin letter, or `U` where none was recorded.

The label

Outcome	Passengers	Share
Did not survive	549	61.6%
Survived	342	38.4%

Mildly imbalanced — nothing like the rare-event problem of the previous lecture, where the positive class was one row in ten. Accuracy will not be absurd here.

✗ It will be something more dangerous than absurd: plausible.

Split first — still

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

assert len(X_train) == 712 and len(X_test) == 179
print(f"train rate {y_train.mean():.4f}   test rate {y_test.mean():.4f}")
# train rate 0.3834   test rate 0.3855
```

Stratified on the **label** this time, not on a predictor: with 179 test rows an unstratified draw can shift the base rate by several points, and every downstream number moves with it.

✓ 712 training passengers. That is a small dataset, and by the end of the lecture the size is doing visible damage.

Before the model: what does *nothing* score?

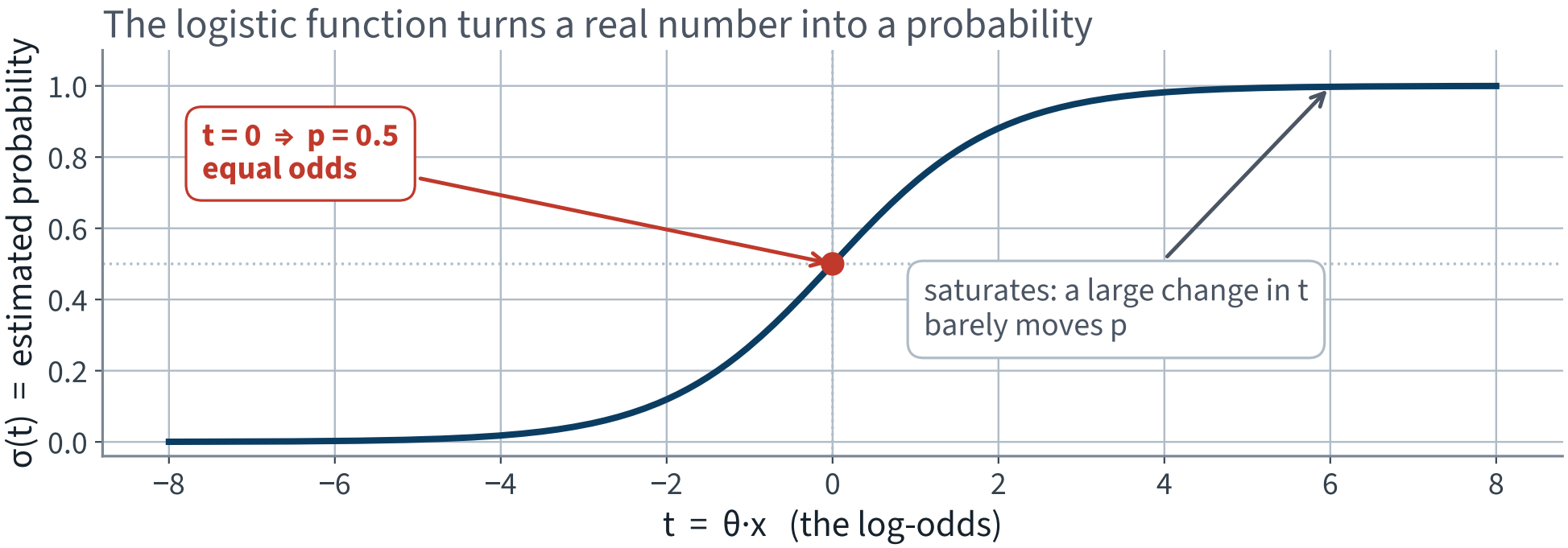
```
p = y_train.mean()                                    # 0.3834

majority_accuracy = 1 - p                             # 0.617
constant_log_loss = -(p*np.log(p) + (1-p)*np.log(1-p))
```

0.666

The log loss of a model that has learned **nothing except the base rate**. It is the entropy of the label, in nats, and every number below it has to be compared against it.

Logistic regression: one function, and the whole difference



$\hat{p} = \sigma(\boldsymbol{\theta}^{\mathsf{T}}\mathbf{x}), \sigma(t) = 1/(1+e^{-t})$: the linear model of least squares, passed through one function. Monotone, so the ranking is the linear model's; saturating, so beyond about $|t| = 5$ more evidence changes almost nothing.

What $\boldsymbol{\theta}^\top \mathbf{x}$ *is*

Invert σ:

$$t = \log \frac{\hat{p}}{1-\hat{p}} = \boldsymbol{\theta}^\top \mathbf{x}$$

The linear model does not predict the probability. It predicts the **log-odds**, and the coefficients act on the odds.

✓ Add 1 to x_j and the odds are multiplied by e^{θ_j}, wherever you started. The probability has no such rule — how far it moves depends on where you started.

What it minimises

$$J(\boldsymbol{\theta}) \;=\; -\frac{1}{m}\sum_{i=1}^{m}\Big[\,y^{(i)}\log\hat{p}^{(i)} + \big(1-y^{(i)}\big)\log\big(1-\hat{p}^{(i)}\big)\Big]$$

THE LOG LOSS, IN NATS

- Only the probability you gave the *true* class enters each term
- Unbounded above: claim 0.001 for a passenger who survived and that term is $-\log 0.001 \approx 6.9$
- It is the negative log-likelihood of the labels under the model, divided by m

Why *this* loss and not something friendlier

A scoring rule is **proper** if the expected score is optimised by reporting your true belief.

- **Log loss** — proper. Reporting anything other than the true p is expected to score worse
- **Brier score**, $\frac{1}{m}\sum(\hat{p}-y)^2$ — also proper, bounded, easier to read; we report it too

> **FAILURE CONDITION — ACCURACY IS NOT PROPER**
>
> Under accuracy a model reporting 0.51 scores exactly the same as one reporting 0.99. So accuracy gives a model *no reason at all* to tell the truth about its uncertainty, and optimising it destroys calibration.

Fit it, and measure it honestly

```
# prep: impute; standardise the numbers (step 4); one-hot the categories
clf = LogisticRegression(C=1e6, max_iter=4000)   # C=1e6 turns the penalty OFF
model = Pipeline([("prep", prep), ("clf", clf)])

cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)
scores = cross_validate(model, X_train, y_train, cv=cv,
                        scoring=["neg_log_loss", "accuracy", "neg_brier_score"],
                        return_train_score=True)
```

`StratifiedKFold`, not `KFold`: with 71 passengers per fold, an unstratified split can hand a fold a base rate several points from the truth.

✗ Scikit-learn regularises logistic regression by default — `C=1.0`. We switch it off so that today's coefficients are the data's and not a compromise with a penalty. Regularisation is the next lecture.

The first numbers

Model	Log loss	Accuracy
Report the base rate to everyone	0.666	61.7%
Logistic regression, 10-fold cross-validated	**0.496 ✓**	**81.5% ✓**

About a quarter below the anchor on log loss, and twenty points of accuracy above the majority class.

Cross-validated, not training. And the per-fold spread of the log loss is **0.22** — larger than any difference this lecture will compare. Read every later gap against it.

Three rows that cannot be right

- `Title_Miss` at −2.81 — but 70.3% of them survived, against 38.4% overall
- `Title_Mrs` at −2.02 — the highest-surviving group in the data
- `Deck_T` is the largest weight in the model — and there is exactly **one** passenger on deck T

✗ These are not weak effects or noisy estimates. They have the wrong sign, and the model's predictions are nonetheless good.

Before reaching for a story about confounding, ask the cheaper question: *what is the shape of the design matrix?*

Count the columns, then count the rank

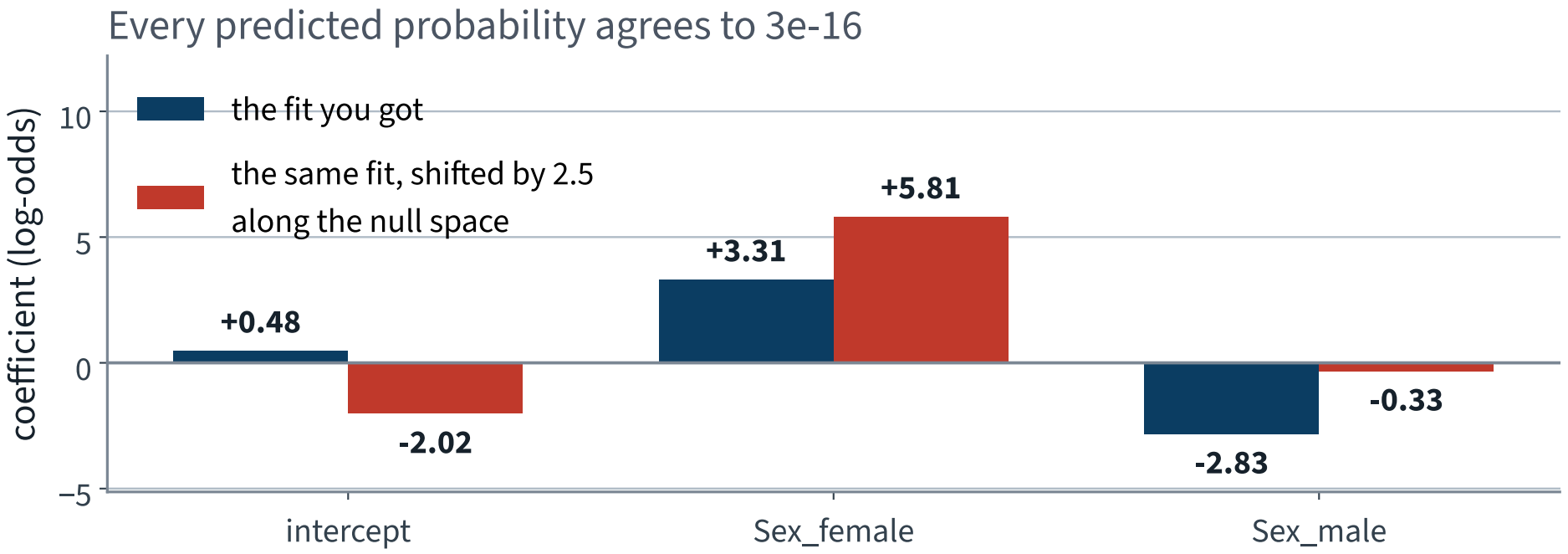
```
Z   = model[:-1].transform(X_train)
Z_b = np.c_[np.ones(len(Z)), Z]          # add the intercept column

print("columns:", Z_b.shape[1])          # 29
print("rank:   ", np.linalg.matrix_rank(Z_b))   # 23
print("cond:   ", np.linalg.cond(Z_b))          # 4.75e+16
```

✗ Six columns of the design matrix are linear combinations of the others.

- **Five from the encoder.** Each one-hot block sums to 1 in every row — and so does the intercept column
- **One from us.** `FamilySize = SibSp + Parch + 1` exactly, for every passenger: an identity, not a correlation

Two coefficient vectors, one model



Add 2.5 to `Sex_female` and to `Sex_male`, subtract 2.5 from the intercept: the two columns sum to the intercept's, so every prediction is unchanged. The minimiser is not unique, and the solver returns one point of a whole family of them. ***✗* A coefficient that can be moved without changing any prediction is not a property of the data.**

The repair: delete the redundancy

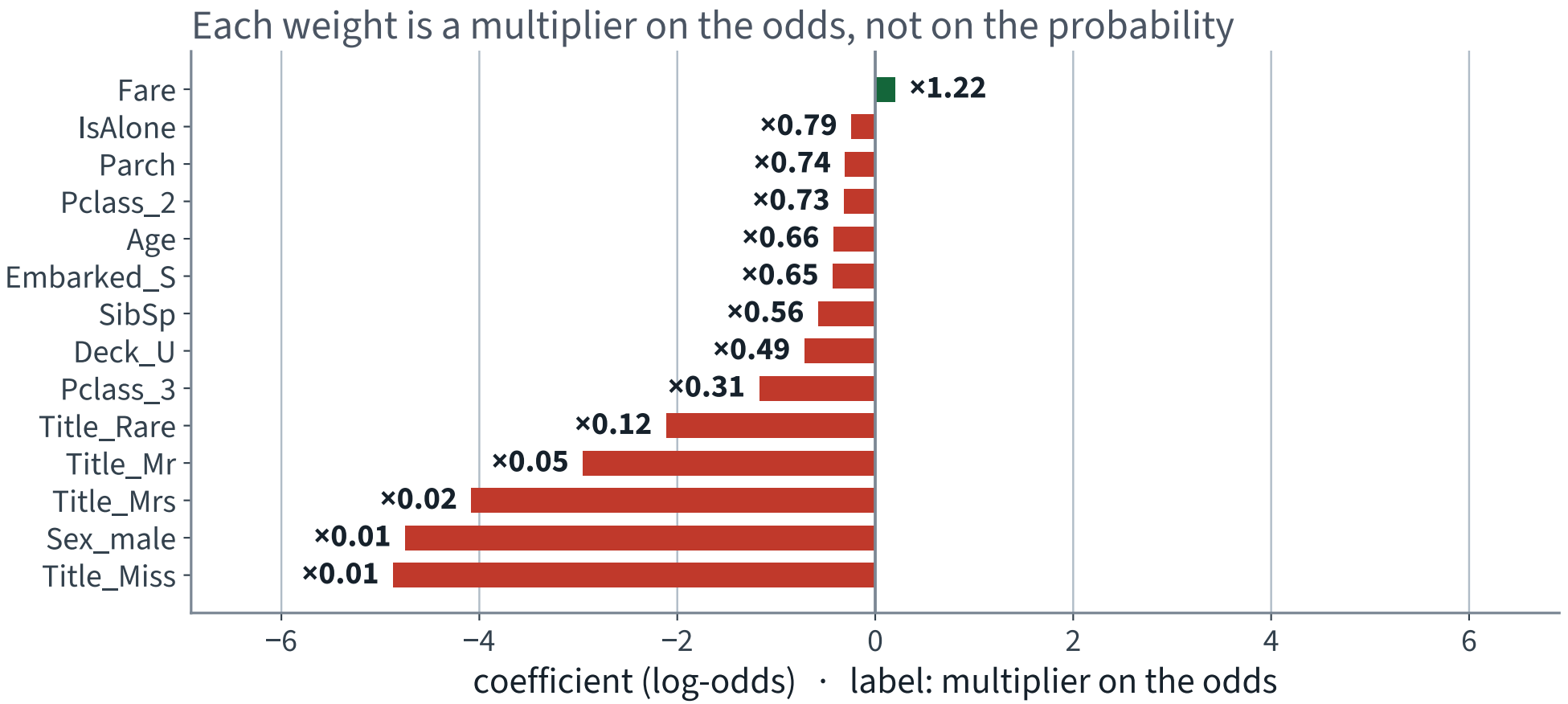
```
NUM = ["Age", "Fare", "SibSp", "Parch"]        # FamilySize removed

("cat", make_pipeline(SimpleImputer(strategy="most_frequent"),
                      OneHotEncoder(drop="first", min_frequency=2,
                                    handle_unknown="infrequent_if_exist")), CAT),
```

Design matrix	Columns	Rank	Condition number
As first written	29	✗ 23	✗ 4.75e+16
One level dropped, FamilySize removed	23	23 ✓	83.6 ✓

Fourteen orders of magnitude. `IsAlone` stays: it is an indicator, not a linear function of the counts.

Now the coefficients can be read



Each bar is a log-odds. The label beside it is what the odds get multiplied by.

Reading one of them out loud

> `Pclass_3` = -1.169
>
> "Holding every other recorded feature fixed, travelling third class multiplies the *odds* of survival by $e^{-1.169} = 0.31$, relative to first class."

- "Holding every other feature fixed" is doing real work — drop it and the sentence is false
- Odds, not probability. Multiplying odds of 1 by 0.31 gives $p = 0.24$; doing it from $p = 0.9$ gives 0.74
- Relative to the reference level — the level `drop="first"` removed, here first class — always

✗ Unique is not the same as *stable*: `Sex` and `Title` still carry much of the same information, so how the effect is split between them moves with the sample. We only fixed uniqueness.

Push the capacity up, and record *both* curves

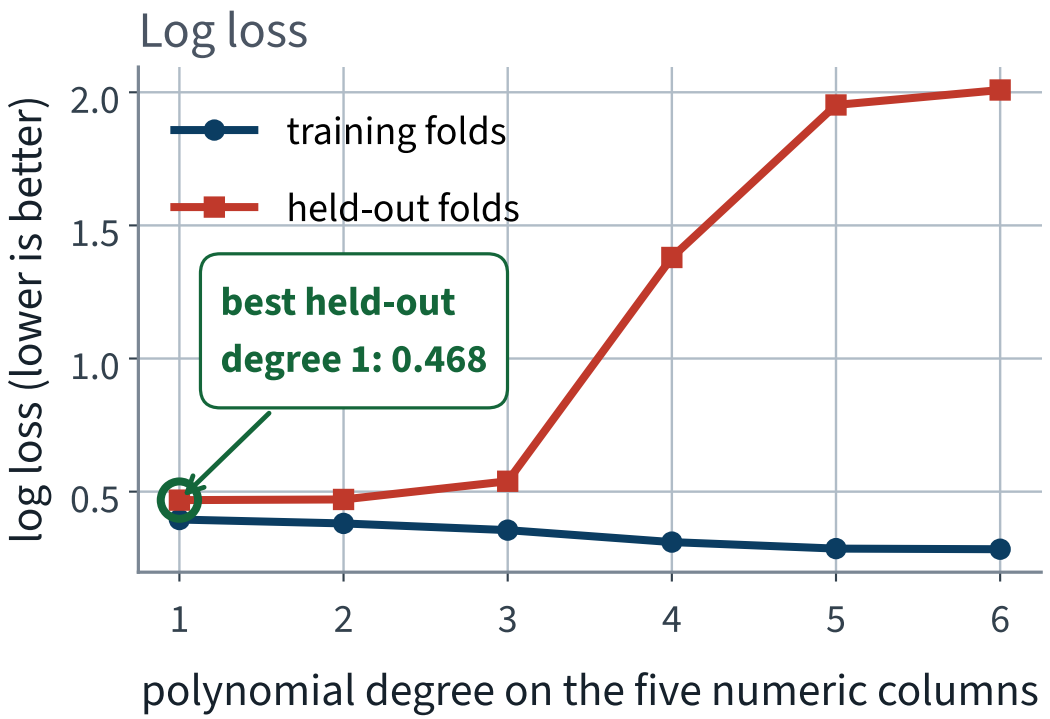
Polynomial features on the four numeric columns only, degrees 1 to 6, the same ten folds:

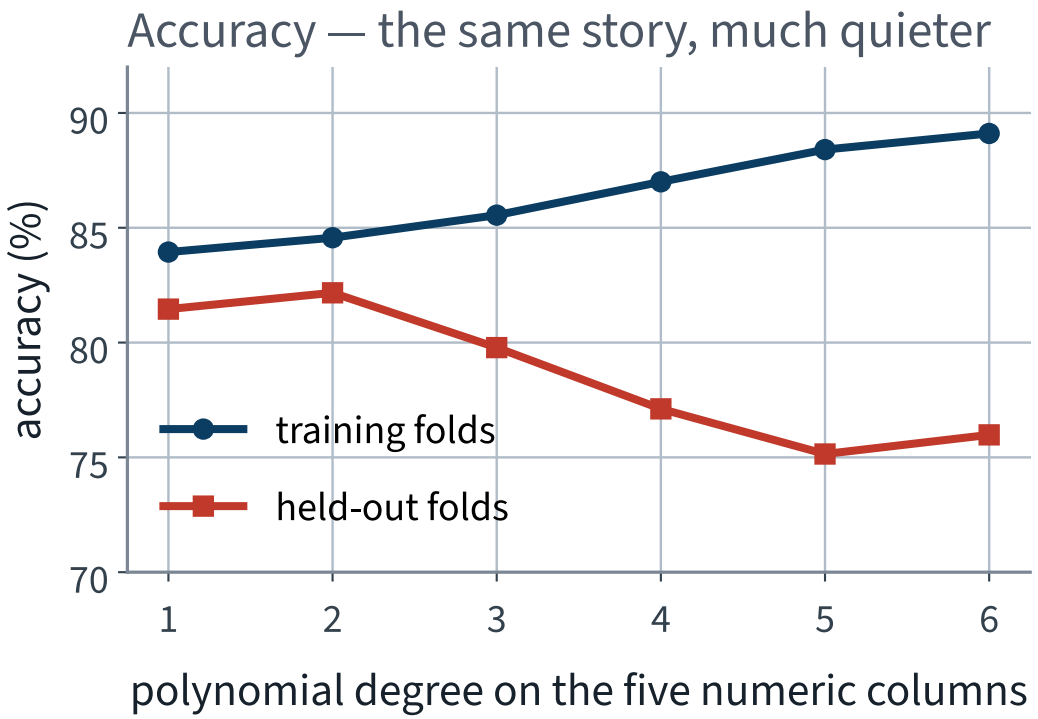
Degree	Columns	Training log loss	Held-out log loss	Held-out accuracy
1	22	0.395	**0.468 ✓**	**81.5% ✓**
2	32	0.381	**0.471 ✓**	**82.2% ✓**
3	52	0.355	0.539	79.8%
4	87	0.310	**✗ 1.380**	77.1%
5	143	0.286	**✗ 1.952**	75.1%
6	227	0.284	**✗ 2.009**	76.0%

Degrees 1 and 2 differ by 0.003 in held-out log loss, and their fold-to-fold spread, 0.13, is 52 times that. **✗ Those two are a tie**, and the curve turns *after* degree 2. Crowning either one — degree 1 has the lower mean — would be reporting noise.

At degree 6 the model has 227 weights to determine from 712 passengers — about three rows per weight.

The two curves





✗ The training curve never stops improving. The held-out curve is flat across degrees 1 and 2, turns after degree 2 and then falls off a cliff.

Log loss saw it first

From degree 2 to degree 5, held-out accuracy falls by 7.0 points. Held-out log loss is multiplied by more than **four**.

- Accuracy only notices when a prediction crosses the cut-off
- Log loss notices a confident prediction becoming a confidently *wrong* one
- The failure is not that the model gets more passengers wrong — it is that it becomes *certain* about the ones it gets wrong

✓ The requirement was for probabilities. The metric that matches the requirement is the metric that saw the damage.

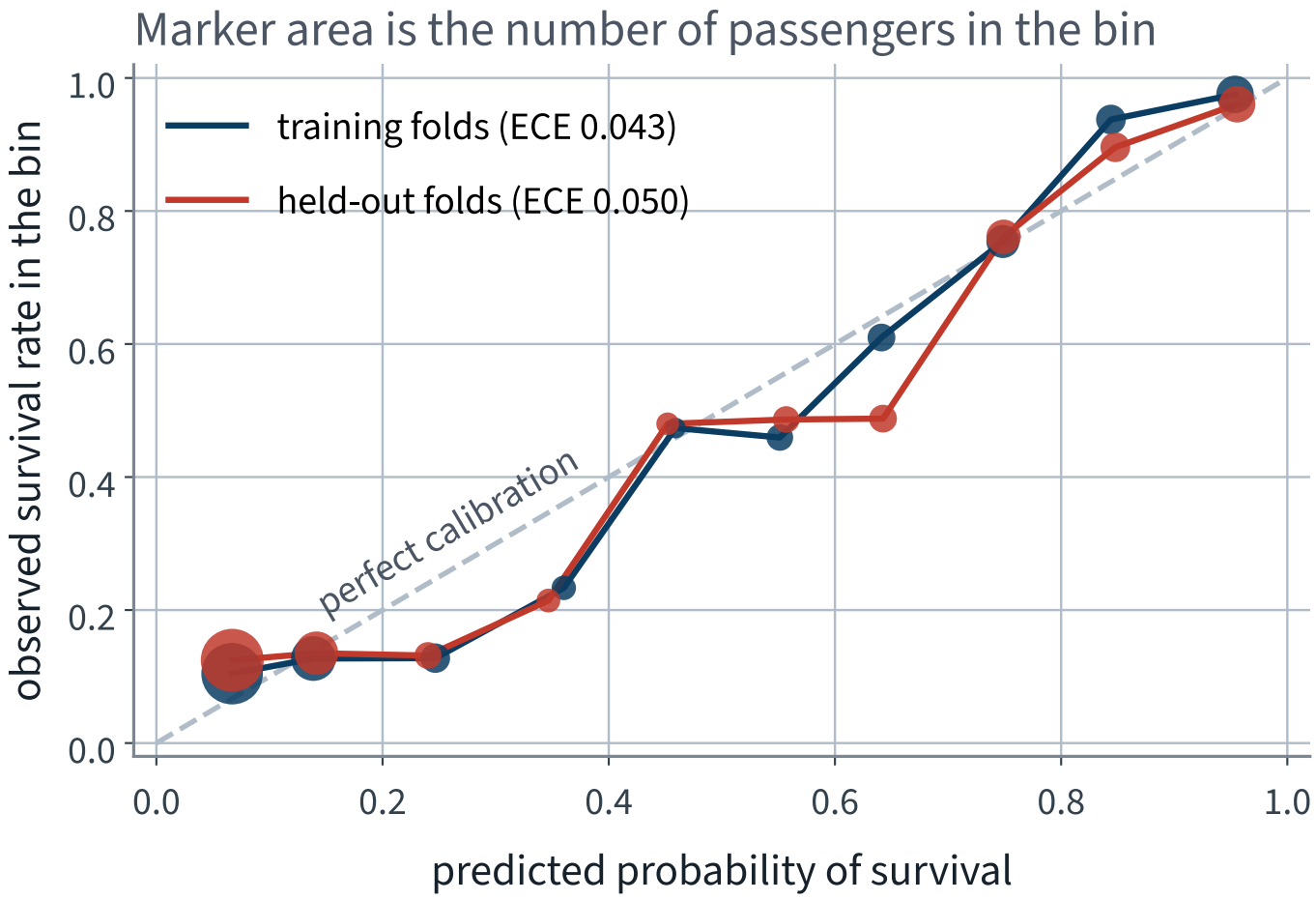
What “calibrated” means, precisely

$$P(y = 1 \mid \hat{p}(\mathbf{x}) = q) \;=\; q \qquad \text{for every } q \in (0, 1)$$

Take every passenger the model scored near q. The fraction who actually survived should be about q.

This is a property of a *set* of predictions, not of any one prediction. You cannot check it on a single passenger, which is exactly why it is so easy to ship a model that fails it.

The reliability diagram



Out of fold — every probability from a fit that never saw that passenger. Close to the diagonal in the bins that hold most of the passengers: the two ends, where the model is confident.

`.predict()` is `.predict_proba() >= 0.5`

FAILURE CONDITION — A BARE LABEL HIDES A DECISION NOBODY MADE

Calling `.predict()` silently applies a cut-off of 0.5, which is cost-optimal only when the two mistakes cost the same. Nothing in the code mentions 0.5. Nothing errors. The accuracy printed is true, and it answers a question the stakeholder did not ask.

Lecture 3 chose an operating point against a stated constraint and defended it. The same discipline applies here, with one addition: **the model produces the probability, and the cut-off is a separate decision taken on top of it**.

Softmax regression

One parameter *vector* per class. Score each class, then normalise:

$$s_k(\mathbf{x}) = \boldsymbol{\theta}_k^\top \mathbf{x}, \qquad \hat{p}_k = \frac{\exp\big(s_k(\mathbf{x})\big)}{\sum_{j=1}^{K} \exp\big(s_j(\mathbf{x})\big)}$$

Every $\hat{p}_k > 0$ and they sum to 1 by construction, so the output is a distribution over the K classes without any further work.

Multi*class*, not multi*label*: softmax assigns one class per instance. Four objects in one photograph need K separate binary classifiers, not this.

Its cost: cross-entropy

$$J(\mathbf{\Theta}) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K} y_k^{(i)} \log \hat{p}_k^{(i)}$$

$y_k^{(i)}$ is 1 if instance i belongs to class k and 0 otherwise, so the inner sum keeps exactly one term: the log of the probability given to the *true* class.

✓ At $K = 2$ this is the log loss, term for term. Logistic regression is the two-class case of softmax, not a different model.

The notebook for this lecture

Open Lecture 4 in Colab

- **Runs on free CPU**
- Gradient descent implemented from the derivation, checked against the closed form to eight decimals, run at three learning rates — one too large, so the cost diverges — and the one-row gradient checked to be unbiased
- Then the Titanic manifest as on the slides: the anchor, the first numbers, the rank computation and the shift that leaves every prediction unchanged, the repair, the coefficients, the degree sweep, the calibration bins, and softmax on three classes

> **WHAT TO CHANGE**
>
> Lower `ETA_DIVERGE` in the learning-rate cell until the cost stops blowing up, and find roughly the largest learning rate that still converges.

Summary

- Descent steps along $-\nabla J$; on the MSE that gradient is $\frac{2}{m}\mathbf{X}^\intercal(\mathbf{X}\boldsymbol{\theta} - y)$ — error times feature — and no matrix is ever inverted
- The learning rate is a hyperparameter, set by watching the cost curve; unscaled features turn the bowl into a thin valley and slow descent down — which is why you scale
- One instance gives an unbiased gradient, which is what makes stochastic and mini-batch descent legitimate rather than merely cheap
- Logistic regression predicts the **log-odds** linearly and is fitted on the log loss, a proper scoring rule; coefficients multiply the odds, always relative to a reference level
- A rank-deficient design leaves the minimiser not unique: the predictions stand, the coefficients mean nothing — count the rank
- A probability is checkable and a label is not — and `.predict()` is a cut-off nobody chose

NOT PRESENTED — FOR AFTERWARDS

Exercises Lecture 4

five, in the style of the written paper

Exercises — Lecture 4

1 Gradient descent on the same data, with the features unscaled, takes many more steps to converge than on scaled features. Explain why, in terms of the shape of the cost surface. [5]

2 On the Titanic manifest, your logistic regression on one-hot encoded columns produces coefficients that look wrong, and the design matrix has rank six less than its column count. State the cause and the standard repair. [5]

3 A model reaches 0.80 accuracy and is *badly calibrated*. Explain what that means, and name a decision it would make wrongly that accuracy cannot see. [4]

Solutions on Lecture 5's deck.

Exercises — Lecture 4 (continued)

4 In a logistic regression with first class as the reference level, the coefficient on `Pclass_3` is -1.169. State what it multiplies, by how much, and why no single change in the survival probability follows from it. [4]

5 State the update rule for batch gradient descent, and say what changes in it for stochastic gradient descent — and what does not. [4]

Solutions on Lecture 5's deck.

THE FIVE SET AT THE END OF LECTURE 3

Solutions Lecture 3

not part of this lecture

Solutions — Lecture 3

1. The normal equation is $\hat{\theta} = (X^\top X)^{-1} X^\top y$. Your training matrix X has a…

✓ $X^\top X$ is singular; the pseudoinverse returns the minimum-norm least-squares solution.

- A linear dependence among columns means X does not have linearly independent columns, so $X^\top X$ does not have full rank and has no inverse
- The fitted *values* are still unique — the projection onto the column space is — only the coefficients are not

2. A pipeline scales the features and then fits a model, and is passed whole to cross_val_score. Explain what…

✓ The scaler would be fitted on every fold's validation rows, so each fold's score could be optimistic — by an amount the code cannot tell you.

- A scaler is a fitted object: its mean and standard deviation are parameters learned from data
- Fitting it before the split lets validation rows influence the transformation applied to themselves

Solutions — Lecture 3 (2)

3. On the MNIST 5-detector, a classifier that never fires scores 90.96% accuracy on the training set. State…

✓ The share of the data that is not a 5.

- With no positive prediction, accuracy is exactly the negative class's base rate
- It measures the data, not the model — which is why it is the anchor every other number is read against

4. Prove or disprove: as the decision threshold is lowered, precision increases monotonically.

✓ Disprove — precision is not monotone in the threshold.

- Lowering the threshold admits one instance at a time; recall cannot fall, but precision rises or falls depending on whether that instance is a true or a false positive
- A single false positive admitted at a high-precision point lowers it, and a later true positive can raise it again

Solutions — Lecture 3 (3)

5. Two models have the same ROC AUC. One is far better on a precision–recall curve. What does that tell…

✓ The positive class is rare.

- ROC uses the false-positive *rate*, whose denominator is the large negative class, so many false positives barely move it
- PR uses precision, whose denominator is what the model predicted, so the same false positives dominate it

NEXT LECTURE

Regularisation, and the bias–variance trade-off

Why the held-out curve turned upward after degree 2, decomposed into three terms — and the one penalty term that gives a fit like today’s rank-deficient one a unique answer